

Cel ćwiczenia

Celem ćwiczenia jest zapoznanie się z mechanizmami ochrony pamięci współdzielonej między wątkami OpenMP i kosztem ich wykorzystania. W tym celu zostaną zaimplementowane różne algorytmy równoległego sortowania kubełkowego, wykorzystujące w różny sposób mechanizmy ochrony współdzielonej pamięci. Wpływ ww mechanizmów na przyspieszenie obliczeń dla problemu sortowania należy ocenić i wyniki badań zamieścić w tworzonej sprawozdaniu.

Wprowadzenie

Równoległe algorytmy sortowania kubełkowego:

1. Każdy wątek czyta całą tablicę początkową. Każdy wątek posiada własne kubełki z ustalonego dla wątku zakresu liczb. Oznacza to że wątek czyta całą tablicę, wybiera z niej liczby z zakresu własnych kubełków i zapisuje do własnych kubełków. Każdy wątek sortuje własne kubełki, a następnie przepisuje dane z tych kubełków do odpowiedniego fragmentu tablicy początkowej (każdy wątek będzie zapisywał do innego fragmentu tablicy).
 - o Odpowiedz na pytania:
 - a) czy potrzebna jest jakaś ochrona danych wspólnych (tablica początkowa: przy odczycie i przy zapisie; kubełki: przy zapisie, sortowaniu kubełka, odczycie)?
 - b) czy zapewnienie by wątki nie czytały tego samego elementu w tym samym czasie przyspieszy program?
 - c) jaki jest rząd złożoności obliczeniowej algorytmu, a jaka jest praca algorytmu równoległego, czy algorytm jest sekwencyjnie-efektywny?
2. Każdy z wątków czyta fragment tablicy przydzielonej tylko dla niego (nie ma kolizji w dostępie do tablicy); przeczytane liczby rozdziela do właściwych kubełków - oznacza to, że może dojść do konfliktu przy zapisie do tego samego kubełka; każdy wątek sortuje przydzielone mu do sortowania kubełki (nie ma konfliktu), a następnie przepisuje dane z tych kubełków do odpowiedniego fragmentu tablicy początkowej (każdy wątek będzie zapisywał do innego fragmentu tablicy).
 - o Odpowiedz na pytania:
 - a) czy potrzebna jest jakaś ochrona danych wspólnych (tablica początkowa: przy odczycie i przy zapisie; kubełki: przy zapisie, sortowaniu kubełka, odczycie)?
 - b) jaki jest rząd złożoności obliczeniowej algorytmu, a jaka jest praca algorytmu równoległego, czy algorytm jest sekwencyjnie-efektywny?
3. Każdy z wątków czyta fragment tablicy przydzielonej tylko dla niego; przeczytane liczby rozdziela do właściwych kubełków. Każdy wątek posiada kubełki dla każdego zakresu liczb (mniejsza pojemność kubełka, ale kubełki mają zakres wszystkich wartości tablicy początkowej). Potrzebny jest dodatkowy krok algorytmu (również równoległy): łączenie kubełków o takim samym zakresie danych, posiadanych do tej pory przez poszczególne wątki (każdy z wątków otrzymuje do połączenia inne kubełki, nie ma konfliktu w dostępie do danych); najpierw połącz (konkatenacja) kubełki o takim samym zakresie, potem je posortuj i przepisuj do tablicy początkowej (oczywiście wszystko równoległe) - taka kolejność gwarantuje, że wielkość sortowanych kubełków będzie taka sama jak w 2. algorytmie.
 - o Odpowiedz na pytania:
 - a) czy potrzebna jest jakaś ochrona danych wspólnych (tablica początkowa: przy odczycie i przy zapisie; kubełki: przy zapisie, łączeniu, sortowaniu kubełka, odczycie)?
 - b) jaki jest rząd złożoności obliczeniowej algorytmu, a jaka jest praca algorytmu równoległego, czy algorytm jest sekwencyjnie-efektywny?
4. Wariant drugi algorytmu (2). Ochrona kubełka powinna zostać zaimplementowana za nie używając locków.
 - o Słowa kluczowe OpenMP: `taskyield`, `atomic`, `compare`, `capture`
 - o [Mechanical Sympathy](#) oraz [prezentacja](#)

Praca algorytmu równoległego oraz pojęcie sekwencyjnie-efektywnego algorytmu opisano na wykładzie:

[MPR-05-Modele-teoretyczne.pdf](#)

Wykonanie ćwiczenia:

1. Pracujemy w zespołach 2-osobowych z poprzednich zajęć, dla uzyskania spójnej implementacji jeśli chodzi o struktury i typy danych, jak i pomiary czasu.
2. Każdy zespół realizuje dwa różne algorytmy opisane we Wprowadzeniu; **osoba z zespołu która zaimplementuje 2 algorytmy otrzymuje trzy punkty więcej (sumarycznie np. osoba implementująca 1 oraz 3 alg. może uzyskać maksymalnie 20+3=23 punkty).**
3. Każdy z algorytmów musi być wyposażony minimum w:
 - o pomiary czasu poszczególnych części algorytmu: (a) losowania liczby - *do wypełnienia tablicy początkowej wykorzystaj równoległy algorytm losowania liczb z poprzednich zajęć*, (b) rozdzielenia liczb do kubełków, (b') w alg. 3 pomiar czasu

połączenia kubeków (może być łącznie z b), (c) sortowania kubeków, (d) przepisania posortowanych kubeków do tablicy początkowej, łącznie z wymianą informacji o wielkości kubeków po rozdzieleniu liczb, dla ustalenia początku przepisywania danych do tablicy początkowej; (e) oraz czas wykonania całości algorytmu (nie wolno sumować, ze względu na sumowanie błędów pomiaru)

- sprawdzenie, czy dane wynikowe są posortowane (poza pomiarem czasu)

4. W kolejnej części badań, w ramach pracy własnej (również w ramach zespołów), porównywane będą właściwości algorytmów, dlatego proszę o pracę w grupach na takich samych strukturach danych, ustalenie takich samych przedziałów liczbowych dla tablicy początkowej, by łatwo i wiarygodnie przeprowadzać później doświadczenia (wyniki porównania będą stanowiły część sprawozdania końcowego).

Wynikiem niniejszego ćwiczenia będą, przesłane przez każdego osobno:

1. plik z kodem zaimplementowanego "swojego" algorytmu (wg przydziału w obrębie swojej grupy),
2. praca własna po zajęciach: przygotowanie wykresu czasu wykonania i przyspieszenia poszczególnych części algorytmu i całości algorytmu na kolejne zajęcia wraz z krótkim opisem algorytmu i jego implementacji (umieścić w sprawozdaniu) - może być już z wykorzystaniem zasobów Cyfronetowych.

W Sprawozdaniu_2, w części dotyczącej niniejszych ćwiczeń, oceniane będą:

1. Poprawność i dobroć wykonanej implementacji - zostanie ona oceniona w oparciu o charakterystyki przyspieszenia oraz kod i jego opis w sprawozdaniu,
2. Wpływ na ocenę sprawozdania będzie miała czytelność kodu: w przypadku braku komentarzy w kodzie programu, jak i nieczytelności kodu (np. brak podziału na funkcje itp.) liczba punktów będzie obniżona, nie więcej niż o 2 pkt (max. za sprawozdanie to 20 pkt.).

Ostatnia modyfikacja: wtorek, 14 kwietnia 2026, 13:55

